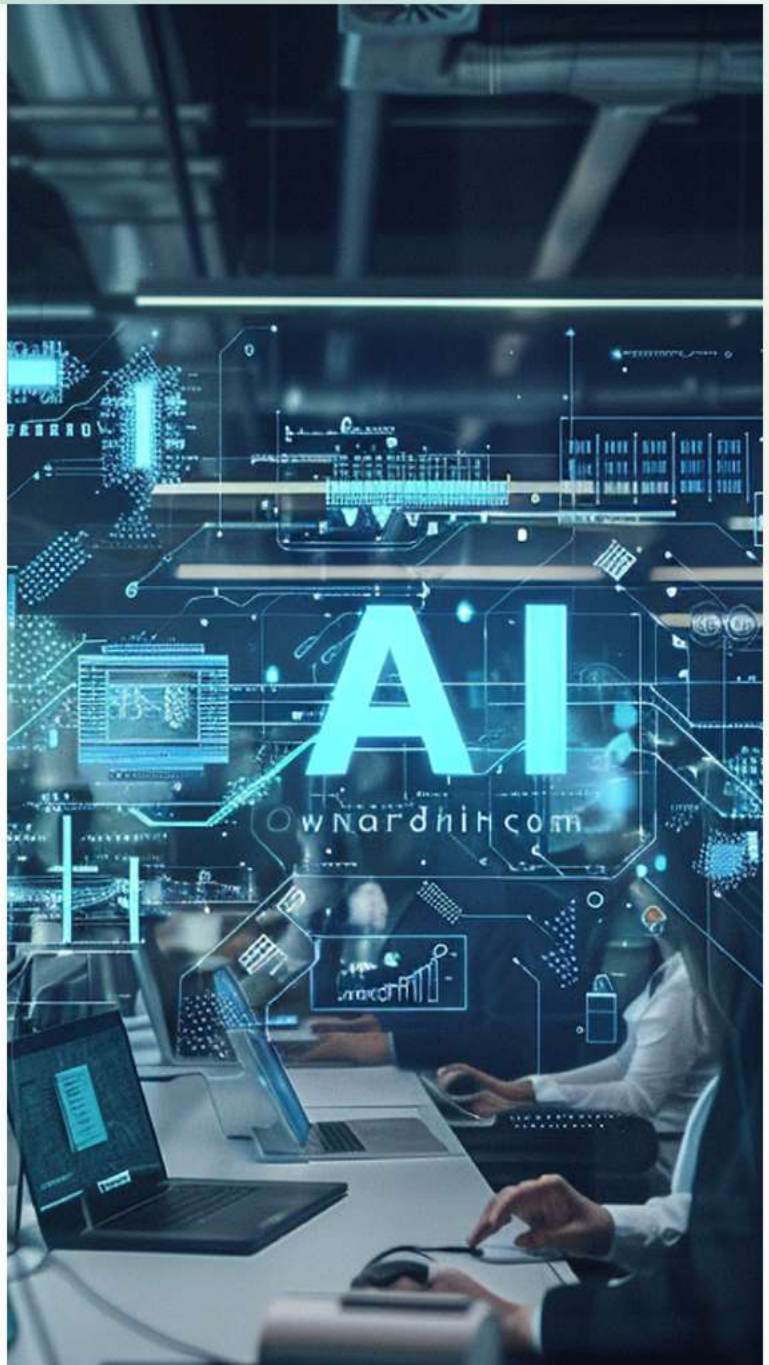
AI-Powered Phishing URL Detection System Report

This report details the design, development, and assessment of an AI-based phishing URL detection system that integrates hybrid techniques such as business logic, real-time threat intelligence, and machine learning to proactively identify phishing attacks.

S.Srinubabu



Proactive Real-Time Phishing URL Detection System

A multi-layered approach integrating AI and threat intelligence APIs

Phishing Threat Overview

Phishing exploits social engineering to steal sensitive data, posing severe cybersecurity risks.

Limitations of Blacklisting

Traditional blacklists react slowly and fail against short-lived phishing websites.

Hybrid Detection Methodology

Combines allow lists, VirusTotal API, and machine learning for comprehensive URL analysis.

Levenshtein Distance Use

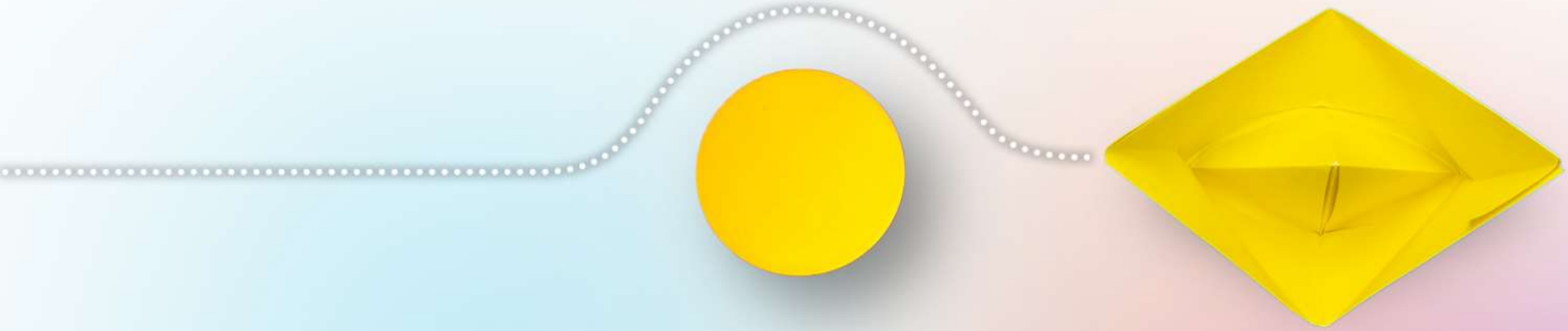
Employs Levenshtein algorithm to detect typosquatting by measuring string similarity.

Machine Learning Model

Random Forest model trained to identify zero-day phishing URLs as backup detection.

System Deployment

Implemented as a Flask-based full-stack web app, achieving over 98% recall accuracy.



Machine Learning for Real-Time Phishing Detection

Proactive identification of phishing threats through URL feature analysis

01

Phishing as a sophisticated cyber-attack

Phishing tricks users into revealing confidential data using fake websites that mimic legitimate services, causing significant financial and personal losses worldwide.

02

Limitations of traditional blacklist methods

Blacklists fail to keep up as attackers rapidly deploy new phishing domains, often active only for a few hours, making reactive measures ineffective.

03

Need for proactive, intelligent detection

Effective defense requires systems that analyze URL characteristics to detect malicious intent, even for previously unseen phishing websites.

04

Machine learning-based detection system

This project develops a model trained on engineered URL features to predict phishing sites, shifting from reactive to predictive cybersecurity.

05

Focus on engineered URL features

Analyzing structural and lexical URL attributes enables early identification of phishing threats before they can cause damage.

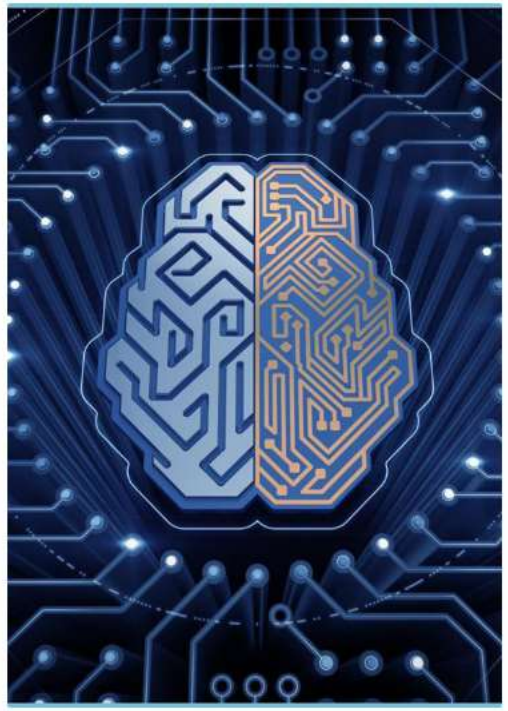
06

Advancing cybersecurity research

Detecting phishing in real-time using intelligent models addresses the growing challenge posed by increasingly sophisticated attacks.

Core Objectives of Phishing Detection Project

Key goals for building an effective real-time URL detection tool



Develop a high-accuracy model for reliable URL classification

Train and select models to detect phishing URLs

Engineer an advanced feature set to identify typosquatting

Design structural and semantic URL features

Build a full-stack web application with real-time analysis

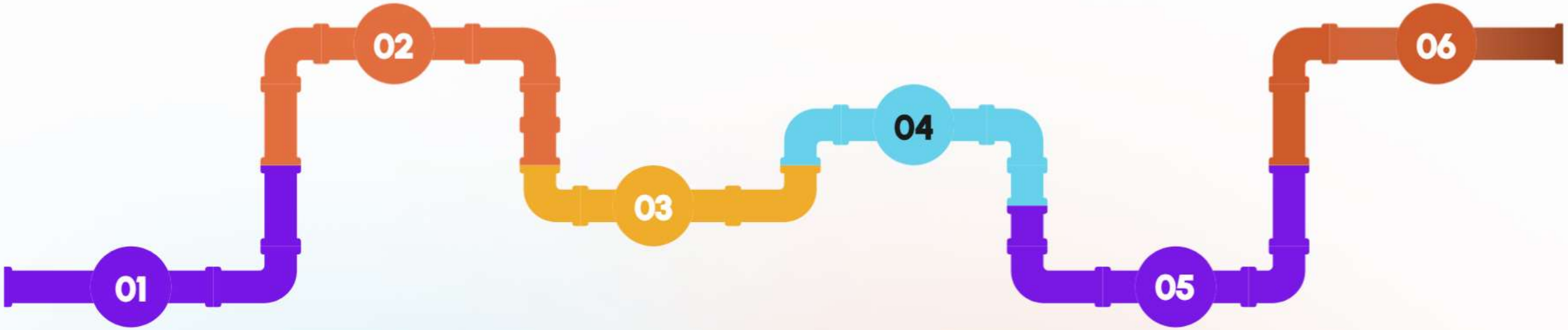
Python Flask backend and HTML/CSS frontend

Integrate VirusTotal API for enhanced threat intelligence

Combine model predictions with external security data

Multi-Layered Phishing Detection Architecture

Sequential decision engine ensuring speed and accuracy



Sequential Decision Engine

Prioritized three-tier logic

Business Logic Rule

Levenshtein distance check

Threat Intelligence API

VirusTotal reputation query

AI Model Prediction

Random Forest probability scoring

False Positive Prevention

Allow list for trusted brands

Efficient Resource Use

Light checks before AI model

Comprehensive Data Acquisition and Preprocessing

Hybrid sourcing and balanced dataset preparation for model training

01

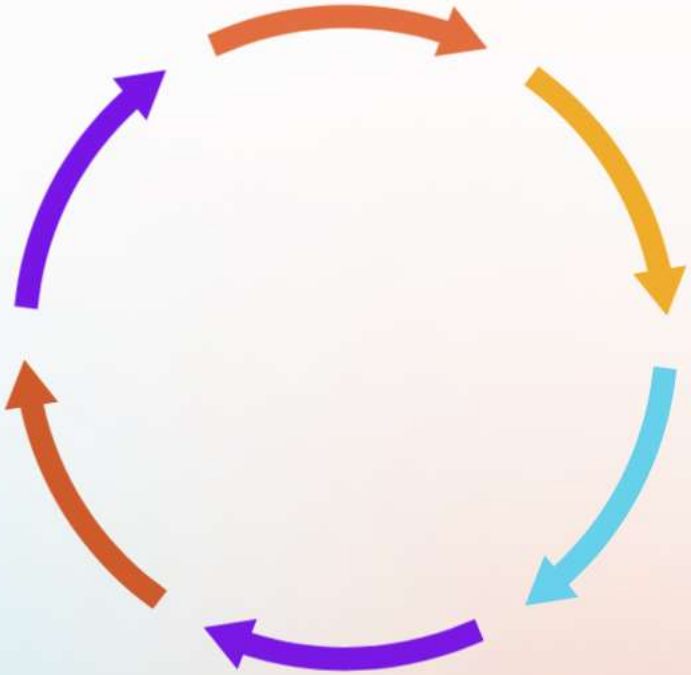
Hybrid sourcing strategy for diverse URL datasets

A combined approach sourcing both legitimate and phishing URLs ensures volume and diversity essential for training robust models.

02

Legitimate URLs sourced from the Tranco list

1 million popular domains from the Tranco list provide a clean and reliable baseline of legitimate URLs for model training.



03

Phishing URLs obtained from Kaggle and PhishTank

Phishing data combines a large static Kaggle dataset with real-time verified URLs from PhishTank to maintain freshness and accuracy.

04

Data cleaning removes duplicates and null values

Preprocessing involves deduplication and null value removal to ensure dataset quality and reliability before model training.

05

Random shuffling ensures balanced dataset distribution

Shuffling the combined dataset creates an even distribution of phishing and legitimate URLs for effective training and testing.

06

Final dataset contains over 320,000 URLs balanced

The resulting super dataset includes approximately 166,000 phishing and 156,000 legitimate URLs, forming a strong foundation for modeling.

Feature Engineering for URL Analysis

Structural and semantic URL features for phishing detection

URL Structural Features

Capture syntactic properties like length, characters, and keyword presence in URLs for analysis.

Binary Flags in URLs

Flags such as IP address presence and '@' symbol indicate suspicious URL traits effectively.

Keyword-Based Detection

Detects login-related keywords within URLs to flag potential phishing attempts early.

Levenshtein Distance Usage

Measures string similarity to identify typosquatting by comparing domains with known brands.

Typosquatting Detection Impact

Low Levenshtein distance values reveal subtle fake domains mimicking trusted brands.

Iterative Model Training and Evaluation

Refining Random Forest with enhanced datasets and features

01 Random Forest chosen for robustness and suitability

Selected for handling high-dimensional data and reducing overfitting.

02 Model training performed iteratively in three versions

Versions V1 to V3 show progressive dataset and feature improvements.

03 V1 trained on initial dataset with basic features

Initial training established baseline model performance.

04 V2 retrained on enriched "super" dataset

Enhanced dataset improved the model's learning capacity.

05 V3 included intelligent Levenshtein distance feature

New feature integration boosted identification accuracy.

06 Final V3 model achieved over 98% recall

High recall ensures minimal false negatives in phishing detection.

Full-Stack Web Application for Phishing Detection

Implementing Flask backend and intuitive HTML frontend interface

Backend server built with Flask framework in Python

Handles loading of the saved phishing model and processes HTTP requests with layered decision logic.

Model integration using saved file phishing_model_v3.joblib

The backend loads and executes the trained model to generate phishing verdicts in real-time.

Frontend created with HTML and CSS for user interaction

Provides a simple input field for URLs and displays results with color-coded confidence scores.

Clear presentation of verdict and confidence scores

Results are shown in an intuitive, color-coded format to enhance user understanding and usability.

Phishing Detection System: Conclusion and Future Enhancements

Summary of outcomes, future improvements, and technology stack used

Project delivers a modern, effective phishing detection system

Combines custom features, robust machine learning, and real-time intelligence to surpass traditional static methods.

Iterative development resolved model's typosquatting blind spot

Addressed typosquatting via the Levenshtein feature, a critical factor driving the project's success and robustness.

Future enhancement includes packaging as a browser extension

Enables automatic, real-time URL scanning before user visits, increasing proactive phishing protection.

Plan to expand system for web content analysis using NLP

Fetch and analyze webpage HTML to detect suspicious text, forms, and scripts beyond URL-based assessment.

Automated retraining pipeline proposed with CI/CD integration

Daily data updates from PhishTank enable continuous model retraining and deployment for adaptive threat response.

Technology stack includes Python, Flask, and Scikit-learn

Utilizes Python for backend, Flask framework, Scikit-learn and Pandas for machine learning and data processing.

Key libraries enhance functionality and data handling

Levenshtein library for string distance, Requests for API calls, Joblib for model storage and serialization.

Development environments support training and deployment

Model training performed in Google Colab, application development using VS Code for efficient workflow.

Thank You